Assignment A1 : Tokenization and Statistical Language Modelling

Individual assignment • 20% individual-report component Released: 19 Nov 2026 Due: 26 Nov 2026, 23:59

Objective. Perplexity is only meaningful *relative to a tokenization*. You will build a tokenizer and an n-gram language model from scratch, then compare their fit to a held-out corpus with a pretrained neural model — and see why per-token perplexity can mislead while per-byte perplexity does not.

Data — a public Kaggle competition. All assignments build on **Jigsaw Unintended Bias in Toxicity Classification** (`jigsaw-unintended-bias-in-toxicity-classification`). Join the competition on Kaggle, then fetch it with the Kaggle API or via `starter/load_data.py` (which uses `kagglehub`). From its `train.csv` we derive, through the starter helpers:

- **Classification** — binary toxicity (`target` ≥ 0.5 = toxic) from `comment_text`, with the **identity columns** (`male`, `female`, `black`, `white`, `christian`, `muslim`, ...) as the **subgroups** for fairness analysis. Helper: `load_textclf()` → train / validation / test splits.
- **Raw text for LM work** — a fixed sample of `comment_text` with a held-out split. Helper: `load_lm()` → (train, held-out) lines.

Models come from HuggingFace, as usual. `check_submission.py` validates your output files before you submit; the grading labels for the `test` split are held out by the teaching team. *Content note: the comments can contain offensive language.*

Scope

Required core: tasks 1–4 (BPE tokenizer, smoothed n-gram LM, per-token & per-byte perplexity, GPT-2 small) and the 4-number output

Optional: the vocab-size sweep (task 5); using a trigram + Kneser–Ney instead of a bigram + add-k.

1. Tasks

1. **Train a BPE tokenizer.** Learn a byte-pair-encoding vocabulary (target *ceiling* ≈8k — a small or repetitive corpus may saturate below this, which is fine) on the LM training text (`load_lm()`) using the `tokenizers` (or `sentencepiece`) library. Report the vocabulary size and the average number of tokens per document.
2. **Build a smoothed n-gram LM.** Implement a **bigram or trigram** model over your BPE tokens with add-k or Kneser–Ney smoothing (your choice; justify it). Train on the LM training split.
3. **Report perplexity two ways** on the held-out split: **per token** and **per byte** (normalise the total negative log-likelihood by the number of tokens, and by the number of UTF-8 bytes, respectively).
4. **Run GPT-2 small locally** (HuggingFace `gpt2`, 124M) and compute the same two numbers on the identical held-out split, using the model's own tokenizer for the token count and raw bytes for the byte count.
5. **[optional] Vocab-size sweep.** Retrain the BPE tokenizer at 2–3 vocabulary sizes and show how per-token perplexity shifts while per-byte perplexity stays roughly stable — direct evidence for the findings prompt.

Runs on your laptop

Everything is CPU-friendly. The n-gram model is pure Python/NumPy. GPT-2 small evaluates perplexity (no training) on the held-out split in a few minutes on a laptop CPU; use a sliding window of 512 tokens. No GPU required.

Deliverable & auto-grading

Submit `a1_perplexities.json` with exactly these keys (floats):

```
{ "ngram_ppl_token", "ngram_ppl_byte", "gpt2_ppl_token", "gpt2_ppl_byte" }
```

Scored against the hidden reference: **per-byte** numbers must fall within a tight tolerance band; **per-token** numbers within a looser band (tokenization-dependent). The grader also re-runs your code on a sanity sample to confirm reproducibility.

Definitions (so your numbers match the reference). Use natural-log NLL and PPL $= \exp(\sum \text{NLL}_{\text{nats}}/N)$, where N is the token count (per *that model's* tokenizer) for per-token PPL, and the number of UTF-8 bytes of the scored text for per-byte PPL. Count bytes identically for both models; token counts are per-tokenizer and need not match.

2. Findings prompt

In your reflection, answer concretely: *why do the per-token numbers mislead when comparing your n-gram model to GPT-2, and why do the per-byte numbers not?* Tie your answer to the vocabularies involved.

3. Reflection (required, max 200 words)

Submit `reflection.md` (or `.pdf`). It must answer these three fixed questions and include the AI-use declaration. The reflection is graded for honesty and insight, not length.

1. **What did you know before?** — your starting point on this topic.
2. **What was hardest, and how did you get unstuck?**
3. **What does your result say, and what would you try next?**

AI-use declaration. State which parts (if any) were generated with AI tools, and *how you verified them.* Using AI is allowed; undisclosed or unverified use is not.

4. Submission

Upload a single `zip` to Brightspace named `A1_<studentnumber>.zip`, containing the required output file(s) listed above, your `code/` folder, and `reflection.md`. Run the format checker in `starter/` on your zip first — it validates file names, schemas, and value ranges locally before you submit. Auto-grading scores your output file against the hidden reference; the format checker passing is a precondition for grading.